

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
"Белгородский государственный технологический университет им. В. Г.
Шухова"
(БГТУ им. В.Г. Шухова)

Институт энергетики, информационных технологий и управляющих систем

Кафедра программного обеспечения вычислительной техники
и автоматизированных систем

Лабораторная работа №3
по дисциплине: Операционные системы
по теме: Файловые системы в ОС Linux (Ubuntu): сравнение,
области эффективности. Виртуальная файловая система.
Пользовательская файловая система

Выполнил: студент группы ПВ-221
Дорошенко Владимир Юрьевич
Проверил: доц. Островский Алексей Мичеславович,
асс. Четвертухин Виктор Романович

Цель работы: Изучить популярные файловые системы в ОС Linux (ext4, Btrfs, ReiserFS, NTFS, FAT32), определить область эффективности каждой из них, разобраться как осуществляется работа с виртуальной файловой системой (VFS) ОС Linux и выполнить разработку пользовательской файловой системы в соответствии с индивидуальным заданием. Цель работы обуславливает постановку и решение следующих задач:

1. Изучить особенности файловых систем ext4, Btrfs, ReiserFS, NTFS, FAT32.
2. Освоить создание, настройку и монтирование файловых систем в ОС Linux (Ubuntu 24.04.1 LTS).
3. Провести экспериментальные измерения на файловых системах ext4, Btrfs, ReiserFS, NTFS и FAT32. Оценить производительность файловых систем по следующим параметрам:
 - Время записи данных в файлы (разного размера).
 - Время чтения данных из файлов (разных размеров).
 - Время выполнения задачи индивидуального задания для разных входных параметров.
4. Изучить основы работы виртуальной файловой системы в ОС Linux, её роль в абстрагировании работы с файловыми системами. Освоить основные механизмы взаимодействия через файловые дескрипторы, а также базовые операции с файловыми системами, реализованные через VFS.
5. Написать программу, в соответствие с индивидуальным заданием №1, реализующую работу алгоритма с большим файлом (размер зависит от конкретного алгоритма, рекомендуемый размер 1–5 ГБ). Язык программирования может быть выбран любой. Обязательное к выполнению задание.
6. Разработать пользовательскую файловую систему в соответствии с индивидуальным заданием №2 с помощью библиотеки FUSE (Filesystem in userspace), которая позволяет разработать и протестировать пользовательские файловые системы без необходимости модификации ядра ОС. Язык программирования для реализации задания: C. Внимание! Выполнение этого задания не является обязательным!
7. Составить отчет с программами, результатами тестов, измерениями, графиками и выводами

Создать виртуальный жесткий диск (например, с помощью dd) объемом 10 ГБ.

```
dd if=/dev/zero of=os_lab3.img bs=1M count=10240
```

Подключаем os_lab3.img как виртуальное устройство с использованием losetup:

```
sudo losetup -fP os_lab3.img
```

Здесь флаги:

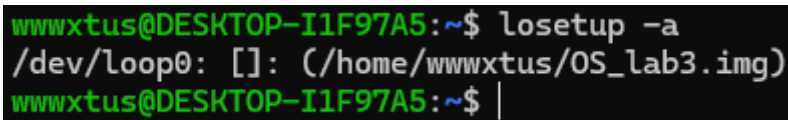
-f: автоматически выберет свободное устройство.

-P: позволит автоматически создать устройства для разделов.

Проверяем, какому loopback - устройству был присвоен диск:

```
losetup -a
```

Пример вывода:



```
wwwxtus@DESKTOP-I1F97A5:~$ losetup -a
/dev/loop0: []: (/home/wwwxtus/OS_lab3.img)
wwwxtus@DESKTOP-I1F97A5:~$ |
```

Команда bash для разбиения диска на 5 разделов для файловых систем ext4, Btrfs, ReiserFS,

NTFS, FAT32:

```
sudo parted /dev/loop0 mklabel gpt
```

```
sudo parted -a optimal /dev/loop0 mkpart primary ext4 0% 2G
```

```
sudo parted -a optimal /dev/loop0 mkpart primary btrfs 2G 4G
```

```
sudo parted -a optimal /dev/loop0 mkpart primary reiserfs 4G 6G
```

```
sudo parted -a optimal /dev/loop0 mkpart primary ntfs 6G 8G
```

```
sudo parted -a optimal /dev/loop0 mkpart primary fat32 8G 10G
```

Создание файловых систем на разделах, форматирование:

```
sudo mkfs.ext4 /dev/loop0p1
```

```
sudo mkfs.btrfs /dev/loop0p2
```

```
sudo mkfs.reiserfs /dev/loop0p3
```

```
sudo mkfs.ntfs /dev/loop0p4
```

```
sudo mkfs.vfat /dev/loop0p5
```

Создание директорий для каждого раздела для подготовки к монтированию:

```
sudo mkdir -p /mnt/ext4 /mnt/btrfs /mnt/reiserfs /mnt/ntfs /mnt/fat32
```

Монтирование:

```
sudo mount /dev/loop0p1 /mnt/ext4
```

```
sudo mount /dev/loop0p2 /mnt/btrfs
```

```
sudo mount /dev/loop0p3 /mnt/reiserfs
```

```
sudo mount -t ntfs /dev/loop0p4 /mnt/ntfs
```

Система FAT32 не поддерживает права доступа POSIX.

Монтирование FAT32 с разрешением записи и чтения:

```
sudo mount -t vfat /dev/loop14p5 /mnt/fat32 -o uid=$(id -u),gid=$(id -g),umask=0022
```

Проверка монтирования:

```
lsblk -f /dev/loop0
```

```
wwwxtus@DESKTOP-I1F97A5:~$ lsblk -f /dev/loop0
```

NAME	FSTYPE	FSVER	LABEL	UUID	FS-AVAIL	FS-USE%	MOUNTPOINTS
loop0							
└─loop0p1	ext4	1.0		431f8bf1-1bb0-4616-a84b-99784534f0b5	1.7G	0%	/mnt/ext4
└─loop0p2	btrfs			cc316381-8ddf-4531-ac15-efbef45656db	1.7G	0%	/mnt/btrfs
└─loop0p3	reiserfs	3.6		cfb4d7d4-4d77-4104-82da-3141423808be			
└─loop0p4	ntfs			6EF2BDF3050A0DB1	1.9G	1%	/mnt/ntfs
└─loop0p5	vfat	FAT32		8429-417B	1.9G	0%	/mnt/fat32

```
wwwxtus@DESKTOP-I1F97A5:~$ |
```

Назначить владельца текущему пользователю

```
sudo chown -R $USER:$USER /mnt/ext4
```

```
sudo chown -R $USER:$USER /mnt/btrfs
```

```
sudo chown -R $USER:$USER /mnt/reiserfs
```

```
sudo chown -R $USER:$USER /mnt/ntfs
```

Установить права доступа для чтения, записи и выполнения

```
sudo chmod -R 755 /mnt/ext4
```

```
sudo chmod -R 755 /mnt/btrfs
```

```
sudo chmod -R 755 /mnt/reiserfs
```

```
sudo chmod -R 755 /mnt/ntfs
```

Установка необходимых пакетов для визуализации (перед тестами):

```
sudo apt update && sudo apt install python3-pip -y
```

```
python3 -m pip install matplotlib --break-system-packages
```

```
sudo apt install python3-pil python3-pil.imagetk python3-tk -y
```

```
import os
import time
import matplotlib.pyplot as plt
from pathlib import Path

MOUNT_POINTS = {
    "ext4": "/mnt/ext4",
    "btrfs": "/mnt/btrfs",
    "ntfs": "/mnt/ntfs",
    "fat32": "/mnt/fat32",
}

import matplotlib
matplotlib.use('TkAgg')

LARGE_FILE_SIZE_MB = 100
SMALL_FILES_COUNT = 1000
SMALL_FILE_SIZE_KB = 10

def write_large_file(directory, size_mb):
    filepath = os.path.join(directory, "large_file.test")
    start_time = time.time()
    with open(filepath, 'wb') as f:
        f.write(b'\0' * (size_mb * 1024 * 1024))
    duration = time.time() - start_time
    os.remove(filepath)
    return duration

def write_small_files(directory, count, size_kb):
    start_time = time.time()
    for i in range(count):
        filepath = os.path.join(directory, f"small_file_{i}.test")
        with open(filepath, 'wb') as f:
            f.write(b'\0' * (size_kb * 1024))
        os.remove(filepath)
    return time.time() - start_time

def read_large_file(directory, size_mb):
    filepath = os.path.join(directory, "large_file.test")
    with open(filepath, 'rb') as f:
```

```

        f.write(b'\0' * (size_mb * 1024 * 1024))
    start_time = time.time()
    with open(filepath, 'rb') as f:
        f.read()
    duration = time.time() - start_time
    os.remove(filepath)
    return duration

def read_small_files(directory, count, size_kb):
    filepaths = []
    for i in range(count):
        filepath = os.path.join(directory, f"small_file_{i}.test")
        with open(filepath, 'wb') as f:
            f.write(b'\0' * (size_kb * 1024))
        filepaths.append(filepath)

    start_time = time.time()

    for filepath in filepaths:
        with open(filepath, 'rb') as f:
            f.read()
        os.remove(filepath)
    return time.time() - start_time

def delete_files(directory, count):
    filepaths = [os.path.join(directory, f"delete_file_{i}.test") for i in
range(count)]
    for filepath in filepaths:
        Path(filepath).touch()
    start_time = time.time()
    for filepath in filepaths:
        os.remove(filepath)
    return time.time() - start_time

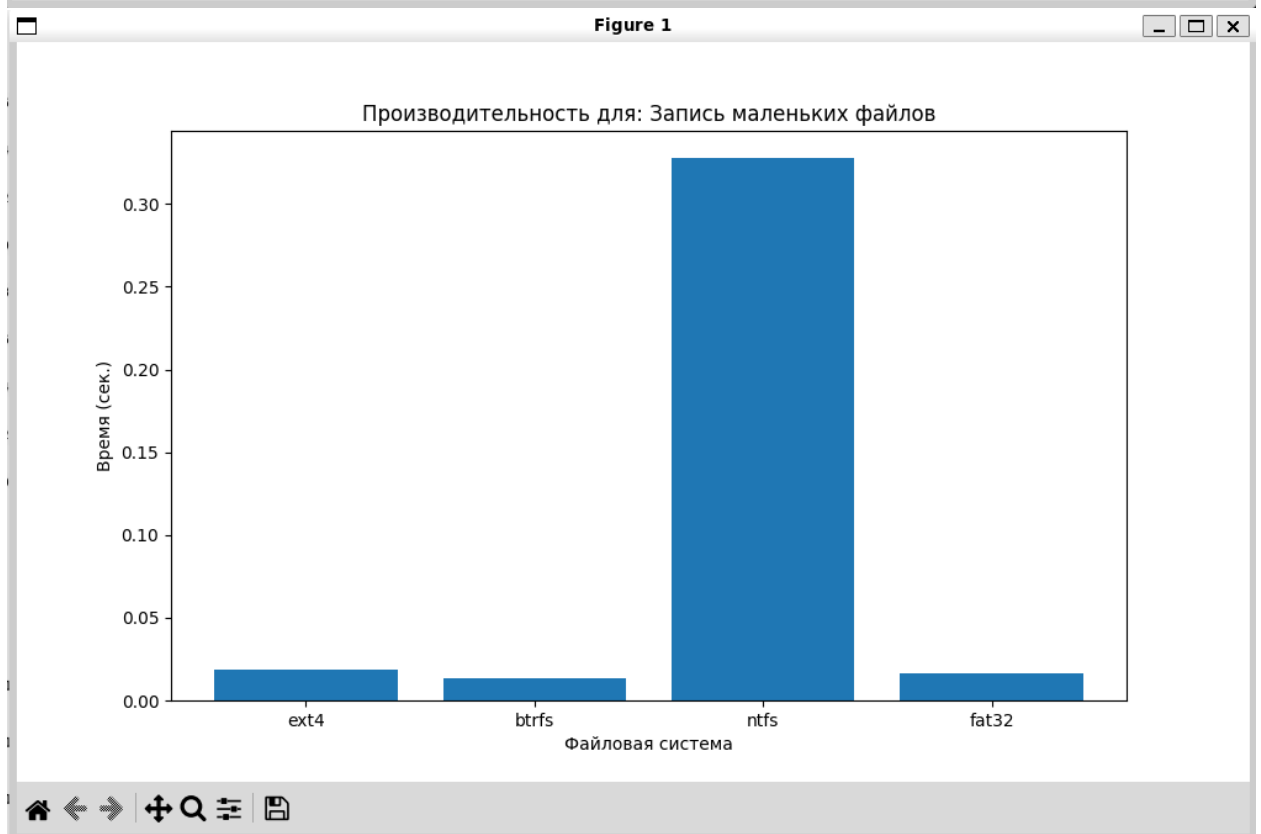
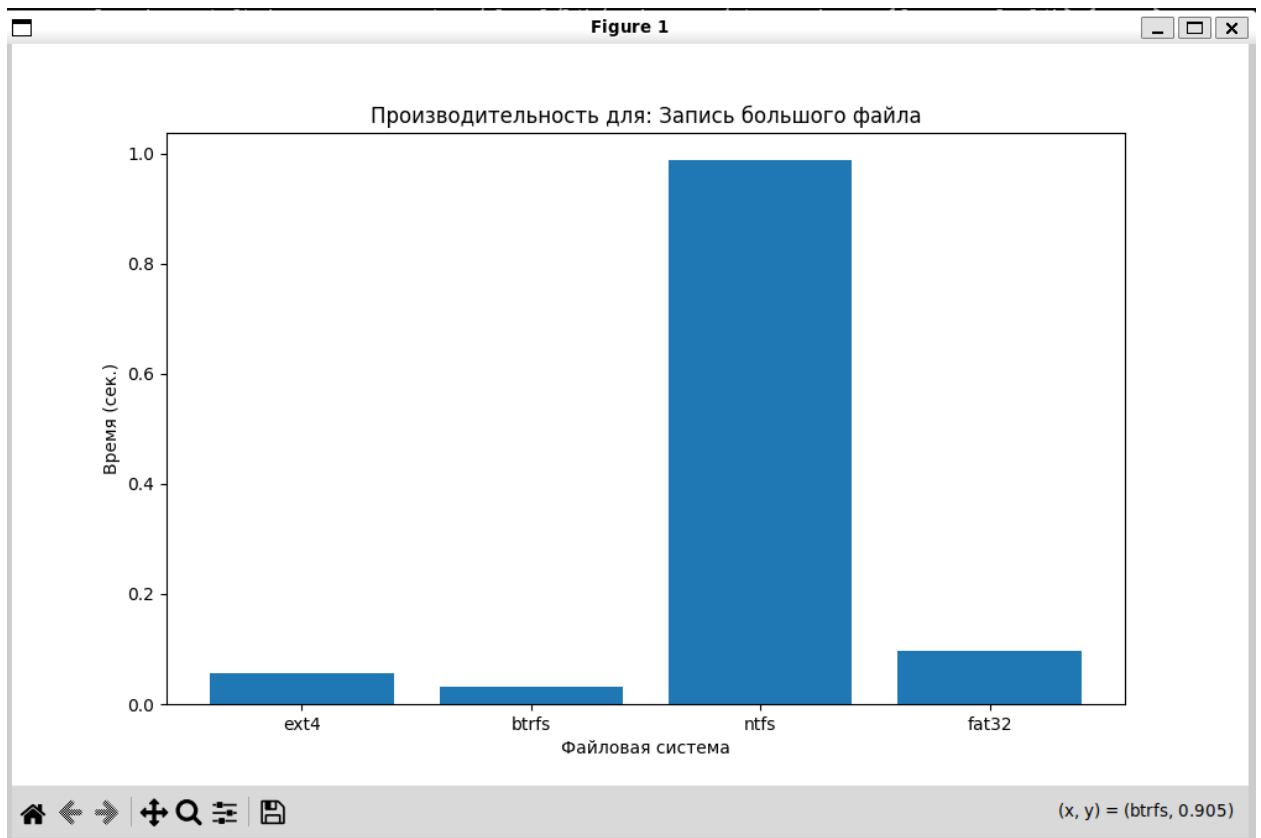
results = {fs: {} for fs in MOUNT_POINTS.keys()}
for fs, path in MOUNT_POINTS.items():
    os.makedirs(path, exist_ok=True)
    results[fs]['Запись большого файла'] = write_large_file(path,
LARGE_FILE_SIZE_MB)
    results[fs]['Запись маленьких файлов'] = write_small_files(path,
SMALL_FILES_COUNT,
SMALL_FILE_SIZE_KB
)

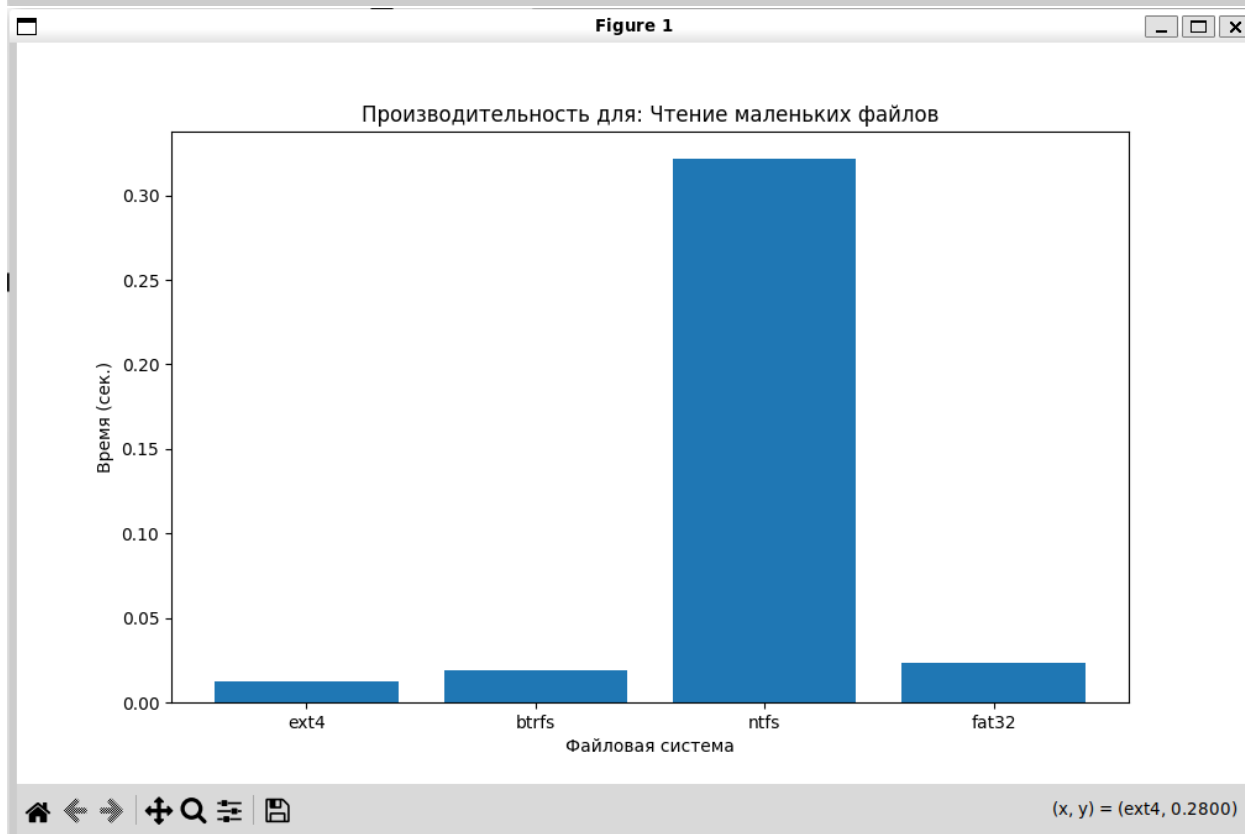
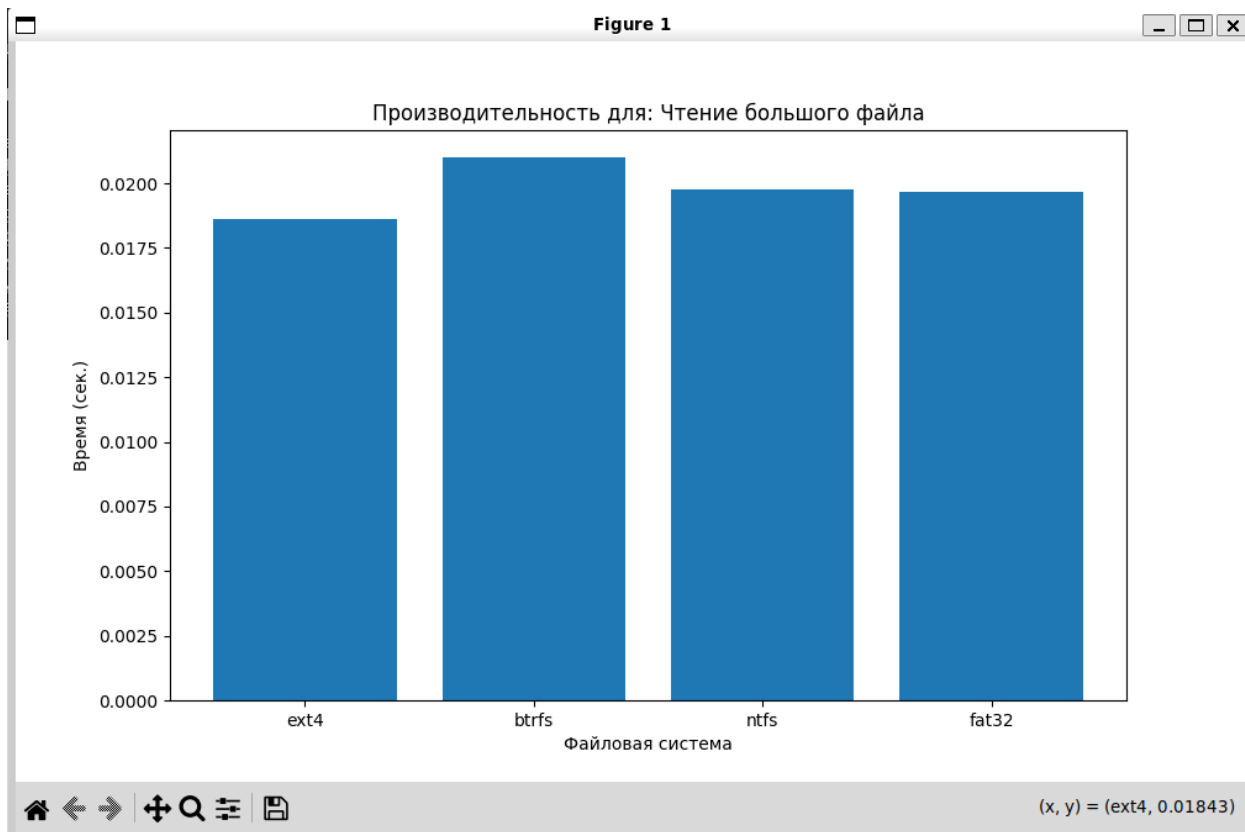
    results[fs]['Чтение большого файла'] = read_large_file(path,
LARGE_FILE_SIZE_MB)
    results[fs]['Чтение маленьких файлов'] = read_small_files(path,
SMALL_FILES_COUNT,

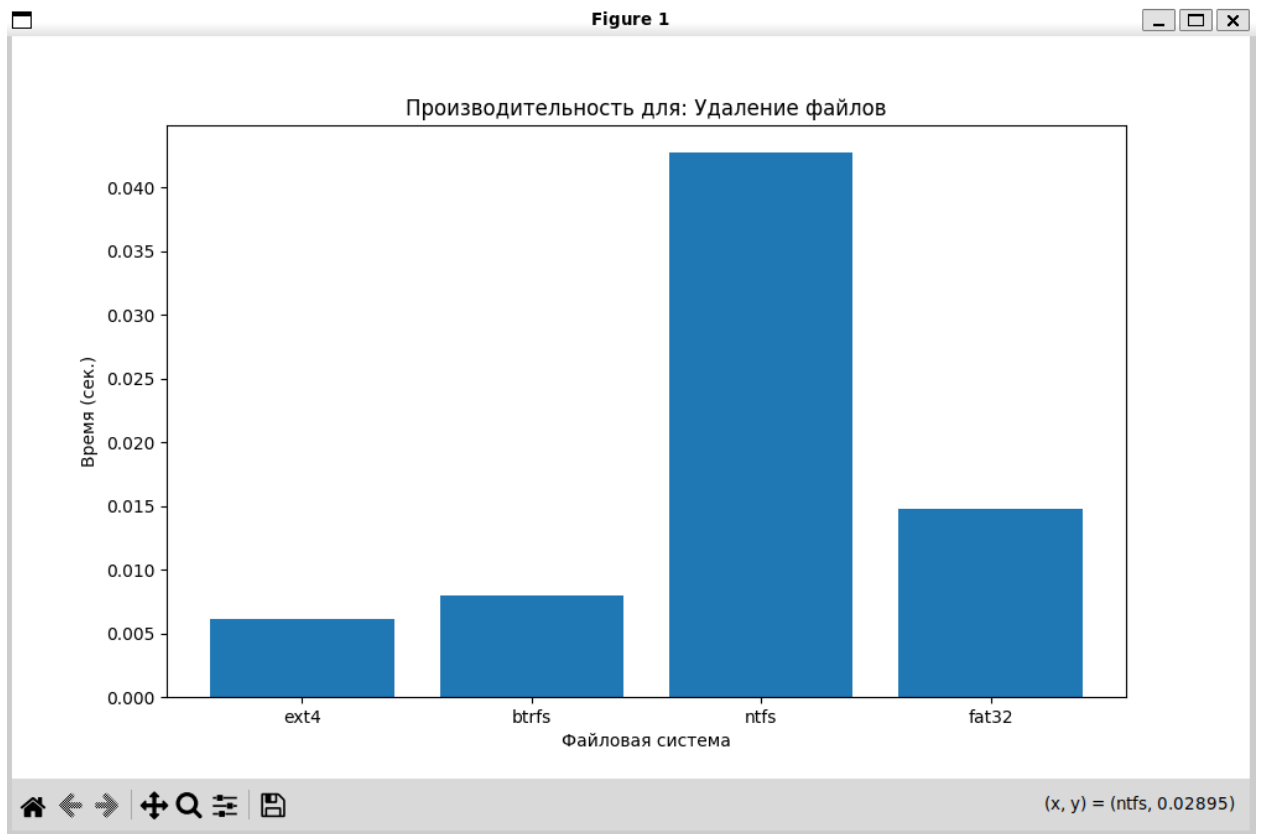
```

```
SMALL_FILE_SIZE_KB)
results[fs]['Удаление файлов'] = delete_files(path, SMALL_FILES_COUNT)

for operation in list(results.values())[0].keys():
    plt.figure(figsize=(10, 6))
    times = [results[fs][operation] for fs in MOUNT_POINTS.keys()]
    plt.bar(MOUNT_POINTS.keys(), times)
    plt.title(f"Производительность для: {operation}")
    plt.xlabel("Файловая система")
    plt.ylabel("Время (сек.)")
    plt.show()
```







8. Поиск подстроки в файле (моделирование нагрузки на чтение файлов). Алгоритм Бойера-Мура.

Сгенерировал тесты в каждой файловой системе с помощью скрипта

```
import os

def create_and_copy_test_file():
    text = (
        "Alice was beginning to get very tired of sitting by her sister on the\n",
        "bank, "\n",
        "and of having nothing to do: once or twice she had peeped into the book\n",
        "her "\n",
        "sister was reading, but it had no pictures or conversations in it, "\n",
        "'and what is the use of a book,' thought Alice, 'without pictures or\n",
        "conversation?'"
    )

    file_systems = {
        "/mnt/ext4/test_file.txt": "ext4",
        "/mnt/btrfs/test_file.txt": "btrfs",
        "/mnt/ntfs/test_file.txt": "ntfs",
        "/mnt/fat32/test_file.txt": "fat32",
    }

    for file_path, fs_type in file_systems.items():
        try:
            os.makedirs(os.path.dirname(file_path), exist_ok=True)

            with open(file_path, "w") as f:
                f.write(text)

            print(f"Файл успешно создан в {fs_type}: {file_path}")
        except Exception as e:
            print(f"Ошибка при создании файла в {fs_type}: {e}")

if __name__ == "__main__":
    create_and_copy_test_file()
```

```
wwwxtus@DESKTOP-I1F97A5:~$ python3 generation_test.py
Файл успешно создан в ext4: /mnt/ext4/test_file.txt
Файл успешно создан в btrfs: /mnt/btrfs/test_file.txt
Файл успешно создан в ntfs: /mnt/ntfs/test_file.txt
Файл успешно создан в fat32: /mnt/fat32/test_file.txt
wwwxtus@DESKTOP-I1F97A5:~$ |
```

Замерил время поиска разных паттернов для каждой файловой системы

```
import os
import time

def boyer_moore_search(text, pattern):
    m, n = len(pattern), len(text)
    if m == 0:
        return []

    shift_table = {char: m for char in set(text)}
    for i in range(m - 1):
        shift_table[pattern[i]] = m - i - 1

    positions = []
    i = 0
    while i <= n - m:
        for j in range(m - 1, -1, -1):
            if pattern[j] != text[i + j]:
                break
        else:
            positions.append(i)
            i += m
            continue
        i += shift_table.get(text[i + m - 1], m)

    return positions

def run_tests():
    text = (
        "Alice was beginning to get very tired of sitting by her sister on the\n",
        "bank, "\n",
        "and of having nothing to do: once or twice she had peeped into the book\n",
        "her "\n",
        "sister was reading, but it had no pictures or conversations in it, "\n",
        "'and what is the use of a book,' thought Alice, 'without pictures or\n",
        "conversation?'"
    )

    test_cases = [
        ("Alice", [0, 260]),
        ("sitting", [41]),
        ("a", [7, 71, 76, 84, 124, 160, 165, 180, 205, 220, 226, 243, 295]),
        ("rabbit", []),
        ("to get very tired", [20]),
        ("alice", []),
        ("conversation?", [288])
    ]

    file_systems = ["/mnt/ext4", "/mnt/btrfs", "/mnt/ntfs", "/mnt/fat32"]
    for fs in file_systems:
```

```

file_path = os.path.join(fs, "test_file.txt")
os.makedirs(os.path.dirname(file_path), exist_ok=True)
with open(file_path, "w") as f:
    f.write(text)

print(f"\nТестирование на файловой системе: {fs}")

for pattern, expected_positions in test_cases:
    with open(file_path, "r") as f:
        content = f.read()
        start_time = time.time()
        positions = boyer_moore_search(content, pattern)
        end_time = time.time()

        elapsed_time = (end_time - start_time) * 1000

        if positions != expected_positions:
            print(f"Ошибка для '{pattern}':")
            print(f"    Ожидается: {expected_positions}")
            print(f"    Получено: {positions}")
            print(f"    Текст вокруг первого совпадения:
{content[positions[0]-10:positions[0]+10] if positions else 'Нет совпадений'}")
            else:
                print(f"Паттерн '{pattern}' найден успешно. Время поиска:
{elapsed_time:.2f} мс")

            assert positions == expected_positions, f"Ошибка для '{pattern}':
{positions} != {expected_positions}"

        print("\nВсе тесты пройдены успешно!")

if __name__ == "__main__":
    run_tests()

```

Результат

```
wwwxtus@DESKTOP-I1F97A5:~$ python3 lab3_2.py
```

```
Тестирование на файловой системе: /mnt/ext4
```

```
Паттерн 'Alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'sitting' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'a' найден успешно. Время поиска: 0.06 мс
```

```
Паттерн 'rabbit' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'to get very tired' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'conversation?' найден успешно. Время поиска: 0.02 мс
```

```
Тестирование на файловой системе: /mnt/btrfs
```

```
Паттерн 'Alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'sitting' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'a' найден успешно. Время поиска: 0.05 мс
```

```
Паттерн 'rabbit' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'to get very tired' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'conversation?' найден успешно. Время поиска: 0.01 мс
```

```
Тестирование на файловой системе: /mnt/ntfs
```

```
Паттерн 'Alice' найден успешно. Время поиска: 0.04 мс
```

```
Паттерн 'sitting' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'a' найден успешно. Время поиска: 0.05 мс
```

```
Паттерн 'rabbit' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'to get very tired' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'conversation?' найден успешно. Время поиска: 0.01 мс
```

```
Тестирование на файловой системе: /mnt/fat32
```

```
Паттерн 'Alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'sitting' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'a' найден успешно. Время поиска: 0.05 мс
```

```
Паттерн 'rabbit' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'to get very tired' найден успешно. Время поиска: 0.01 мс
```

```
Паттерн 'alice' найден успешно. Время поиска: 0.02 мс
```

```
Паттерн 'conversation?' найден успешно. Время поиска: 0.01 мс
```

```
Все тесты пройдены успешно!
```

```
wwwxtus@DESKTOP-I1F97A5:~$ |
```

Быстрее всего обрабатывает FAT32

Вывод: Изучил популярные файловые системы в ОС Linux (ext4, Btrfs, ReiserFS, NTFS, FAT32), разобрался как осуществляется работа с виртуальной файловой системой (VFS) ОС Linux и выполнил индивидуальное задание